

Project Documentation

Store Manager: keep track of Inventory

Introduction

Project Title: Store manager: keep track of Inventory

Team ID: NM2025TMID36367

Team Leader:

Name- Priyadarshini. A

Mail id -priyadarshini2553@gmail.com

Team Members:

1.Abinaya R – abinayaanu1922@gmail.com

2.Anusree V– vanusree58@gmail.com

3.Keerthana T-keerthanaashwini3@gmail.com

4.Raashidha Aafreen B-aafreen6835@gmail.com

2. Project Overview

Purpose:

The project aims to build a smart Inventory Management System that keeps stores organized and efficient. It empowers businesses to track, update, and manage stock in real time with ease. With features such as automated alerts, product search, and sales records, it minimizes effort and errors. Ultimately, it delivers a seamless store experience and supports smarter decision-making. The system enhances productivity by reducing manual work and saving valuable time. It provides accurate insights into sales and stock levels

for better business planning. Overall, the project contributes to effective store management and improved customer satisfaction.

Features:

- Inventory Management – Maintains accurate stock levels and ensures timely updates.
- Cart & Checkout – Supports adding products to cart, clearing on checkout, and recording sales.
- Product Management – Enables adding new products with details like name, price, stock, and tags.
- Sales Records – Stores complete details of sales including products, value, date, and time.

3. Architecture

1. Frontend

- **Technology:** React.js
- **UI Frameworks:** Bootstrap & Material UI
- **Purpose:**
 - Provides a responsive and interactive user interface.
 - Handles user interactions such as login, project posting, bidding, chat, and feedback.
 - Communicates with the backend through RESTful APIs.

2. Backend

- **Technology:** Node.js with Express.js
- **Purpose:**
 - Manages server logic and API endpoints.
 - Handles authentication, authorization, inventory operations, and project management.
 - Acts as a bridge between the frontend and database.

3. Database

- **Technology:** MongoDB (NoSQL Database)
- **Purpose:**
 - Stores user data, inventory details, project information, bids, applications, chat messages, and feedback.
 - Provides scalability and flexibility for handling unstructured data.

Component Structure

•The project follows a component-based architecture, where each feature is divided into small, reusable units.

• Cart Module

- Cart.jsx → Main cart component.
- CartItem.jsx → Represents each item in the cart.

• Inventory Module

- Inventory.jsx → Displays inventory overview.
- Product.jsx → Single product details.

• Product Module

- Product.jsx → Core product structure.
- ProductCatalog.jsx → Shows the entire product catalog.
- ProductList.jsx → Displays product listings.
- Sales Module
 - Sales.jsx → Handles sales functionality.
 - SaleRecord.jsx → Displays individual sale records.

• Common Components

- AddProduct.jsx → Add new products to inventory.

- NavBar.jsx → Navigation bar for switching between modules.

- Layout.jsx → Defines overall page layout.

Routing

- The project uses React Router.

Routes:

- /inventory → Shows inventory (Inventory.jsx, ProductList .jsx).

- /cart → Displays cart items (Cart.jsx, CartItem.jsx).

- /sales → Shows sales history (Sales.jsx, SaleRecord.jsx).

- /add-product → Add new products (AddProduct.jsx).

4.Set up and instructions

Prerequisites:

- Node.js and npm

- React.js

- HTML, CSS, and JavaScript.

installation:

Follow the steps below to set up the application on your local system:

Start backend:

cd server

npm start

Start frontend:

cd client

npm start

- Frontend: <http://localhost:3000>
- Backend API: <http://localhost:5000>

5. Folder Structure

Client

store-manager-inventory/

|

|—— client/ # Frontend (React.js)

| |—— public/ # Static files (index.html, icons, etc.)

| |—— src/

| | |—— components/ # Reusable UI components

| | |—— pages/ # Application pages (Dashboard, Login, etc.)

| | |—— services/ # API calls / Axios services

| | |—— context/ # React Context for state management

| | |—— App.js # Root component

| | |—— index.js # Entry point

	└── package.json	# Frontend dependencies
	└── server/	# Backend (Node.js + Express.js)
	└── config/	# Database & environment configurations
	└── controllers/	# Request handlers (business logic)
	└── models/	# Mongoose models (User, Project, Chat, etc.)
	└── routes/	# Express route definitions
	└── middleware/	# Auth & security middleware
	└── server.js	# Server entry point
	└── package.json	# Backend dependencies
	└── .gitignore	# Ignored files for Git
	└── README.md	# Project documentation
	└── package.json	# Root package (if using concurrently)

6. Running the application

Frontend :

```
cd client
npm start
```

- Runs the React.js application.
- Accessible at:

<http://localhost:3000>

Backend :

cd server

npm start

- Runs the Node.js + Express.js server.
- Default API endpoint:

<http://localhost:5000>

Accessing the Application

- Open your browser and go to:

<http://localhost:3000>

7. Authentication

1. JWT-Based Authentication

- On successful login, the server generates a **JWT token**.
- The token is returned to the client and stored securely (local storage or cookies).
- Ensures only authenticated users can access features like **inventory management, project posting, bidding, feedback, and chat**

2. Middleware Protection

- A custom **authentication middleware** validates the JWT token before granting access.
- Public routes: Register, Login

8. User interface

- Dashboard Overview – Shows key inventory stats like total products, stock levels, and low stock alerts.

- Product Management – Easy access to view, add, edit, or delete products.

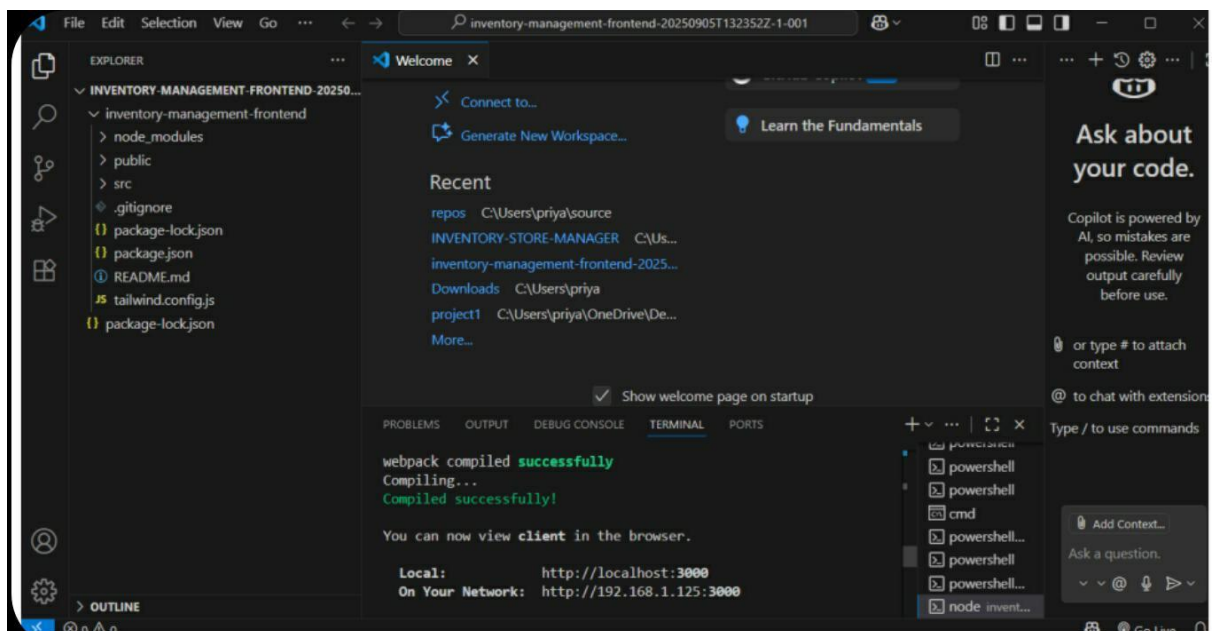
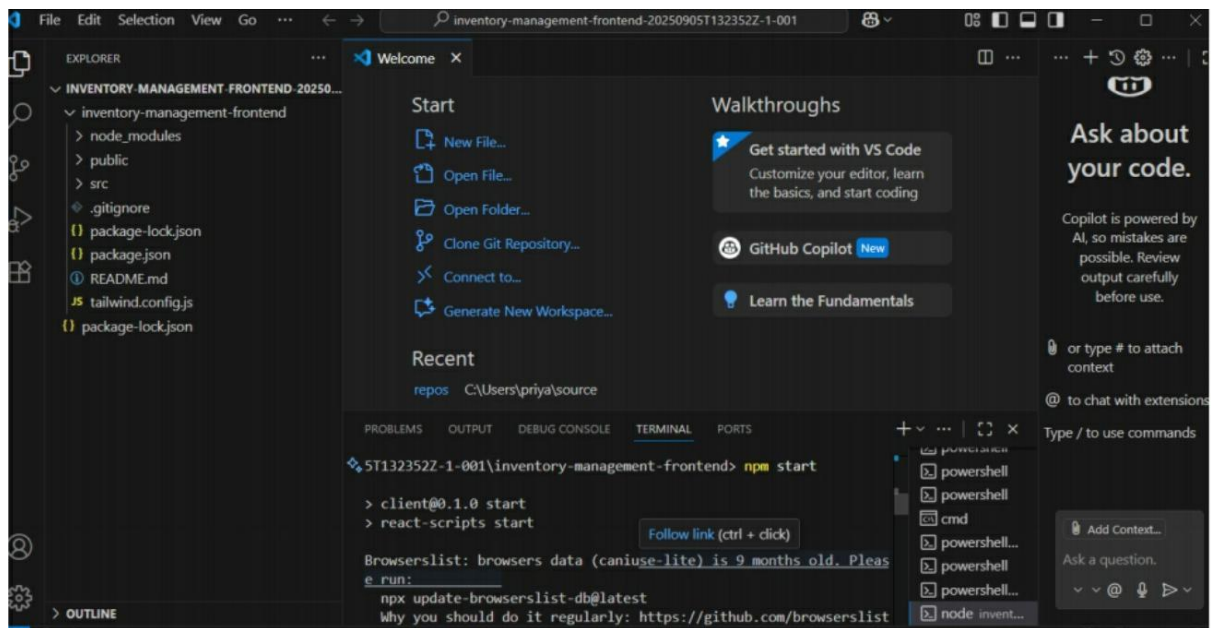
- Search & Navigation – Quick search, filters, and clear navigation for smooth inventory tracking.

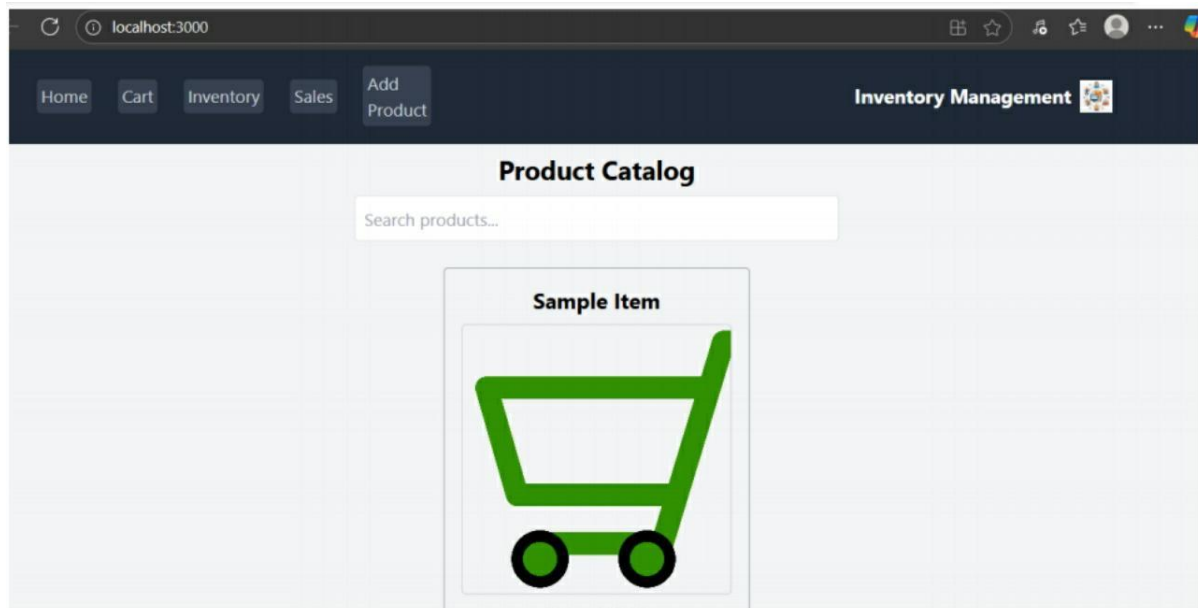
9. Testing

- Login & User Access – Test that only authorized users (manager or staff) can log in and access the system.

- Product & Stock Management – Verify that adding, editing, deleting products, and updating stock levels work correctly.

10. Screenshots or Demo





11. known issues

- Limited Error Handling – Some error messages may not be detailed enough for users to understand the issue.
- Performance with Large Data – The system may slow down when handling a very large number of products.
- No Offline Support – The application requires an internet connection; it won't work offline.

Real-Time Updates – Inventory changes and chat messages are not fully real-time (require page refresh or manual fetch).

12. Future enhancement

- Offline Mode – Allow the system to work without internet and sync data when online.
- Multi-Store Support – Manage inventory across multiple branches or locations from one system.
- **Real-Time Features** – Implement WebSockets (Socket.io) for instant inventory updates, live chat, and bid notifications.

- **Mobile Application** – Develop Android/iOS apps for store managers and freelancers to manage tasks on the go.
- **Barcode/QR Code Integration** – Scan products directly to update or track inventory quickly.
- **Payment Gateway Integration** – Enable secure payments between managers and freelancers inside the platform.
- **AI Recommendations** – Suggest inventory restocking levels and freelancer-project matches based on past data.